

Vel Tech Group of Institutions

Name: GOPICHAND KAKIVAI

Email: vtu29748@veltech.edu.in

Roll no: vtu29748

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L3

VTU_DAA_Week 9_PAH

Attempt : 1

Total Mark : 50

Marks Obtained : 39.5

Section 1 : Coding

1. Problem Statement

Diya is given a weighted, undirected graph with N vertices. She wants to find the minimum weight Hamiltonian cycle in the graph, if one exists. A Hamiltonian cycle is a cycle that visits every vertex exactly once, except for the starting and ending vertices, which are the same. The weight of a Hamiltonian cycle is the sum of the weights of all edges in the cycle:

$$W = w(v_1, v_2) + w(v_2, v_3) + \dots + w(v_N, v_1)$$

Write a program using Backtracking to find the minimum weight Hamiltonian cycle.

Example: For a graph with 4 vertices and the adjacency matrix:

0 10 15 20

10 0 35 25

15 35 0 30

20 25 30 0

The algorithm explores all Hamiltonian cycles starting from vertex 0.

Among all valid cycles, 0 1 3 2 0 has the minimum weight: $10 + 25 + 30 + 15 = 80$.

Input Format

The first line contains an integer N representing the number of vertices in the graph.

The next N lines each contain N space-separated integers representing the adjacency matrix of the weighted graph. The value at $\text{graph}[i][j]$ is the weight of the edge between vertices i and j. A value of 0 means no edge exists. The matrix is symmetric ($\text{graph}[i][j] == \text{graph}[j][i]$) and the diagonal is always 0 ($\text{graph}[i][i] == 0$).

Output Format

If a minimum weight Hamiltonian cycle exists, print Minimum Weight Hamiltonian Cycle: on the first line, then print the vertices of the cycle in square brackets separated by a comma and a space, including the starting vertex at both the beginning and the end of the cycle. On the next line, print Total Weight: followed by a space and the integer total weight.

If no Hamiltonian cycle exists, print No Hamiltonian Cycle found with minimum weight.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4

0 10 15 20

10 0 35 25

15 35 0 30

20 25 30 0

Output: Minimum Weight Hamiltonian Cycle:

[0, 1, 3, 2, 0]

Total Weight: 80

Answer

```
#include <stdio.h>
```

```
#include <limits.h>
```

```
#include <stdbool.h>
```

```
int n;
```

```
int g[15][15];
```

```
int path[15], best[15];
```

```
bool vis[15];
```

```
int bestCost = INT_MAX;
```

```
bool found = false;
```

```
void dfs(int pos, int cost) {
```

```
    if(cost >= bestCost) return;
```

```
    if(pos == n) {
```

```
        if(g[path[n-1]][0] != 0) {
```

```
            int total = cost + g[path[n-1]][0];
```

```
            if(total < bestCost) {
```

```
                bestCost = total;
```

```
                found = true;
```

```
                for(int i = 0; i < n; i++)
```

```
                    best[i] = path[i];
```

```
            }
```

```
        }
```

```
    }
```

```
    return;
```

```
}
```

```
for(int v = 1; v < n; v++) {
```

```
    if(!vis[v] && g[path[pos-1]][v] != 0) {
```

```
        vis[v] = true;
```

```
        path[pos] = v;
```

```
        dfs(pos + 1, cost + g[path[pos-1]][v]);
```

```
        vis[v] = false;
```

```

    }
}

int main() {
    scanf("%d", &n);

    for(int i = 0; i < n; i++)
        for(int j = 0; j < n; j++)
            scanf("%d", &g[i][j]);

    path[0] = 0;
    vis[0] = true;

    dfs(1, 0);

    if(!found) {
        printf("No Hamiltonian Cycle found with minimum weight");
        return 0;
    }

    printf("Minimum Weight Hamiltonian Cycle:\n");
    printf("[");
    for(int i = 0; i < n; i++)
        printf("%d, ", best[i]);
    printf("0]\n");

    printf("Total Weight: %d", bestCost);

    return 0;
}

```

Status : Partially correct

Marks : 8.5/10

2. Problem Statement

Alice is tasked with determining whether a given graph contains a Hamiltonian Cycle and, if it does, listing all such cycles. A Hamiltonian Cycle in a graph is a cycle that visits each vertex exactly once and returns to the starting vertex.

Write a program using Backtracking that takes a graph represented by an adjacency matrix and finds all Hamiltonian Cycles starting from the first vertex (vertex 0), if any exist.

Input Format

The first line contains an integer N representing the number of vertices in the graph.

The next N lines each contain a single string – the name of each vertex (no surrounding spaces).

The following N lines each contain N space-separated integers (0 or 1) representing the adjacency matrix. A value of 1 at position $[i][j]$ means an edge exists between vertex i and vertex j; 0 means no edge.

Output Format

For each Hamiltonian Cycle found, print a line in the format: `['v0', 'v1', ..., 'vn', 'v0']` where vertex names are enclosed in single quotes, separated by a comma and a space, with the starting vertex repeated at the end. Print cycles in the order they are discovered by the backtracking algorithm.

If no Hamiltonian Cycle exists, print: No Hamiltonian Cycle

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 8

a
b
c
d
e
f
g

```
h
0 1 0 0 0 0 0 1
1 0 1 0 0 0 0 0
0 1 0 1 0 0 0 1
0 0 1 0 1 0 1 0
0 0 0 1 0 1 0 0
0 0 0 0 1 0 1 0
0 0 0 1 0 1 0 1
1 0 1 0 0 0 1 0
```

Output: ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'a']
['a', 'h', 'g', 'f', 'e', 'd', 'c', 'b', 'a']

Answer

```
#include <stdio.h>
#include <string.h>
#include <stdbool.h>
```

```
int n;
int adj[10][10];
char name[10][15];
```

```
int path[10];
bool vis[10];
int found = 0;
```

```
void printCycle() {
    printf("[%s", name[path[0]]);
    for(int i = 1; i < n; i++) {
        printf(", %s", name[path[i]]);
    }
    printf(", %s]\n", name[path[0]]);
    found = 1;
}
```

```
void dfs(int pos) {
    if(pos == n) {
        if(adj[path[n-1]][path[0]] == 1) {
            printCycle();
        }
        return;
    }
}
```

```

    for(int v = 1; v < n; v++) {
        if(!vis[v] && adj[path[pos-1]][v] == 1) {
            vis[v] = true;
            path[pos] = v;

            dfs(pos + 1);

            vis[v] = false;
        }
    }
}

int main() {
    scanf("%d", &n);

    for(int i = 0; i < n; i++) {
        scanf("%s", name[i]);
    }

    for(int i = 0; i < n; i++)
        for(int j = 0; j < n; j++)
            scanf("%d", &adj[i][j]);

    path[0] = 0;
    vis[0] = true;

    dfs(1);

    if(!found)
        printf("No Hamiltonian Cycle");

    return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

A company has N employees, and each employee can contribute a fixed number of training hours.

The manager wants to form groups of employees such that the total training hours of a selected group is exactly equal to the required training goal S.

Write a program using Backtracking to find all distinct subsets of employee training hours whose sum is exactly equal to S.

Each employee's training hours can be used only once.

If the input contains duplicate hour values, the output must not contain repeated subsets.

Condition

A subset is valid if:

$$h_1 + h_2 + \dots + h_k = S$$

Input Format

The first line contains an integer N representing the number of employees.

The second line contains N space-separated integers representing the training hours of each employee. The input may contain duplicate values.

The third line contains an integer S representing the required training goal.

Output Format

Print each distinct valid subset on a separate line. Within each subset, print elements in ascending order, each followed by a space (including the last element). Print subsets in the order they are found during backtracking on the sorted input.

If no valid subset exists, print: No subset found

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6
2 2 3 5 6 8
10

Output: 2 2 6
2 3 5
2 8

Answer

```
#include <stdio.h>
#include <stdlib.h>
```

```
int n, S;
int arr[25];
int subset[25];
int found = 0;
```

```
void printSubset(int size) {
    for(int i = 0; i < size; i++)
        printf("%d ", subset[i]);
    printf("\n");
}
```

```
void solve(int idx, int sum, int size) {
    if(sum == S) {
        found = 1;
        printSubset(size);
        return;
    }
```

```
    if(idx == n || sum > S)
        return;
```

```
    int prev = -1;
```

```
    for(int i = idx; i < n; i++) {
        if(arr[i] == prev) continue;
```

```
        subset[size] = arr[i];
        solve(i + 1, sum + arr[i], size + 1);
```

```
        prev = arr[i];
    }
```

```
}
```

```
int main() {
```

```

scanf("%d", &n);

for(int i = 0; i < n; i++)
    scanf("%d", &arr[i]);

scanf("%d", &S);

for(int i = 0; i < n - 1; i++) {
    for(int j = i + 1; j < n; j++) {
        if(arr[i] > arr[j]) {
            int t = arr[i];
            arr[i] = arr[j];
            arr[j] = t;
        }
    }
}

solve(0, 0, 0);

if(!found)
    printf("No subset found");

return 0;
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Nikila is exploring the N Queens Problem. This classic problem involves placing N queens on an $N \times N$ chessboard such that no two queens threaten each other. In chess, queens can move horizontally, vertically, and diagonally, so the challenge is to arrange the queens to ensure they do not share the same row, column, or diagonal.

Write a program using Backtracking to find and print all valid solutions to the N Queens Problem.

Input Format

The input consists of a single integer N representing both the size of the

chessboard (N×N) and the number of queens to be placed.

Output Format

Print all valid solutions in the order they are found by the backtracking algorithm. Each solution is represented as an N×N matrix where 1 indicates the position of a queen and 0 represents an empty cell. Each value is followed by a space. Each row is printed on a new line. Print a blank line after each solution including the last one. If no valid solution exists, print nothing.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4

Output: 0 0 1 0

1 0 0 0

0 0 0 1

0 1 0 0

0 1 0 0

0 0 0 1

1 0 0 0

0 0 1 0

Answer

```
#include <stdio.h>
#include <stdbool.h>
```

```
int n;
int board[10];
bool col[10], d1[20], d2[20];
int found = 0;
```

```
void printBoard() {
    for(int i = 0; i < n; i++) {
        for(int j = 0; j < n; j++) {
            if(board[i] == j) printf("1 ");
            else printf("0 ");
        }
    }
}
```

```

        printf("\n");
    }
    printf("\n");
    found = 1;
}

void solve(int row) {
    if(row == n) {
        printBoard();
        return;
    }

    for(int c = 0; c < n; c++) {
        if(!col[c] && !d1[row - c + n] && !d2[row + c]) {
            board[row] = c;
            col[c] = d1[row - c + n] = d2[row + c] = true;

            solve(row + 1);

            col[c] = d1[row - c + n] = d2[row + c] = false;
        }
    }
}

int main() {
    scanf("%d", &n);

    solve(0);

    return 0;
}

```

Status : Partially correct

Marks : 1/10

5. Problem Statement

Sarah wants to verify whether a given arrangement of queens on a 4×4 chessboard constitutes a valid solution to the 4 Queens Problem. The objective is to position four queens on the board in a way that prevents any queen from threatening another, considering horizontal, vertical, and

diagonal movements.

To determine the validity of the solution, Sarah needs to ensure that there are exactly four queens on the board, each row and column contains only one queen, and no two queens share the same diagonal.

Your task is to assist Sarah by implementing a program that checks these criteria for the given chessboard arrangement.

Input Format

The input consists of 4 lines, each containing 4 space-separated integers representing one row of the chessboard. Each integer is either 1 (queen present) or 0 (empty cell).

Output Format

Print a single line indicating whether the placement is valid, checking conditions in the following priority order:

1. If the total number of queens is not exactly 4: False - Invalid number of queens: X where X is the count of queens on the board.
2. If a row does not contain exactly one queen: False - Row Y has Z queens where Y is the 0-based row index and Z is the number of queens in that row.
3. If a column does not contain exactly one queen: False - Column Y has Z queens where Y is the 0-based column index and Z is the number of queens in that column.
4. If a main diagonal (top-left to bottom-right) contains more than one queen: False - Main diagonal ($d=X$) has Y queens where X is the diagonal identifier computed as column - row (range -3 to 3) and Y is the number of queens on that diagonal.
5. If an anti-diagonal (top-right to bottom-left) contains more than one queen: False - Anti-diagonal ($d=X$) has Y queens where X is the diagonal identifier computed as column + row (range 0 to 6) and Y is the number of queens on that diagonal.
6. If all conditions are satisfied: True - Valid 4 Queens solution: All conditions satisfied

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 0 1 0 0

0 0 0 1

1 0 0 0

0 0 1 0

Output: True - Valid 4 Queens solution: All conditions satisfied

Answer

```
#include <stdio.h>
```

```
int main() {
    int a[4][4];
    int row[4] = {0}, col[4] = {0};
    int d1[10] = {0}, d2[10] = {0};
    int total = 0;

    for(int i = 0; i < 4; i++) {
        for(int j = 0; j < 4; j++) {
            scanf("%d", &a[i][j]);
            if(a[i][j] == 1) {
                total++;
                row[i]++;
                col[j]++;
                d1[j - i + 3]++;
                d2[i + j]++;
            }
        }
    }

    if(total != 4) {
        printf("False - Invalid number of queens: %d", total);
        return 0;
    }

    for(int i = 0; i < 4; i++) {
        if(row[i] != 1) {
            printf("False - Row %d has %d queens", i, row[i]);
            return 0;
        }
    }

    for(int j = 0; j < 4; j++) {
        if(col[j] != 1) {
```

```

        printf("False - Column %d has %d queens", j, col[j]);
        return 0;
    }
}

for(int d = 0; d < 10; d++) {
    if(d1[d] > 1) {
        printf("False - Main diagonal (d=%d) has %d queens", d - 3, d1[d]);
        return 0;
    }
}

for(int d = 0; d < 10; d++) {
    if(d2[d] > 1) {
        printf("False - Anti-diagonal (d=%d) has %d queens", d, d2[d]);
        return 0;
    }
}

printf("True - Valid 4 Queens solution: All conditions satisfied");
return 0;
}

```

Status : Correct

Marks : 10/10